```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import pandas as pd
df = pd.read_csv("/content/diabetes_dataset.csv")
```

```
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps: Generate code with df | New interactive sheet

```
df.tail()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outc
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
# To find Zero values present in each selected columns
columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in columns:
    print(col, ":", (df[col] == 0).sum())
```

```
Glucose : 5
BloodPressure : 35
SkinThickness : 227
Insulin : 374
BMI : 11
```

```
# To convert the zeroes values inti NaN
import numpy as np
columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
df[columns] = df[columns].replace(0,np.nan)
```

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
# Median Imputation

df[columns].median()
```

	0
Glucose	117.0
BloodPressure	72.0
SkinThickness	29.0
Insulin	125.0
BMI	32.3

dtype: float64

```
# To fill the missing values
for col in columns:
    df[col] = df[col].fillna(df[col].median())
```

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
# Train - Split - Test

from sklearn.model_selection import train_test_split

# Separate input and output
x = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

```
# Split the dataset
x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# Check shapes
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(614, 8)
(154, 8)
(614,)
(154,)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

```
print(type(x_train))
print(type(x_test))
print(x_train.shape)
print(x_test.shape)
```

```
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
(614, 8)
(154, 8)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    class_weight="balanced",
    random_state=42
)
```

```
model.fit(x_train, y_train)
```

```
LogisticRegression
LogisticRegression(class_weight='balanced', random_state=42)
```

```
y_pred = model.predict(x_test)

print("Predicted:", y_pred[:10])
print("Actual:", y_test[:10].values)
```

```
Predicted: [1 0 0 0 0 0 1 1 0 1]
Actual: [0 0 0 1 0 0 1 1 0 0]
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
Accuracy: 0.7337662337662337
Precision: 0.6031746031746031
Recall: 0.7037037037037037
F1 Score: 0.6495726495726496
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

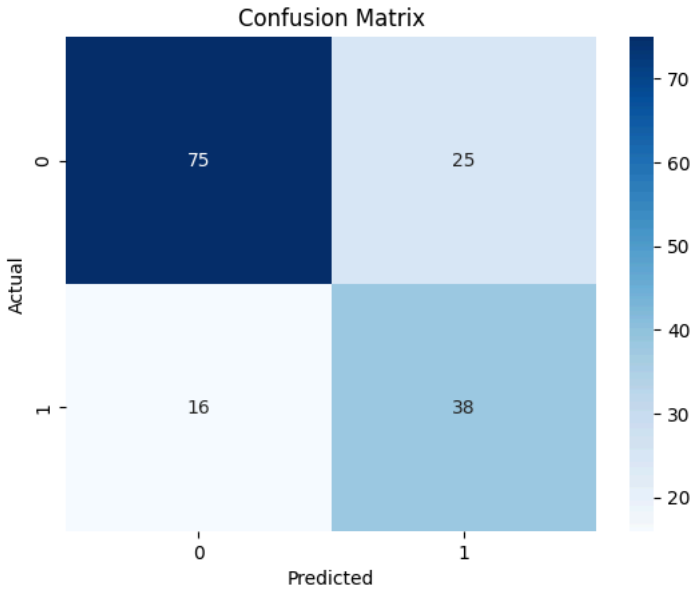
print(cm)
```

```
[[75 25]
 [16 38]]
```

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
from sklearn.metrics import roc_auc_score, roc_curve

y_prob = model.predict_proba(x_test)[:, 1]

auc = roc_auc_score(y_test, y_prob)

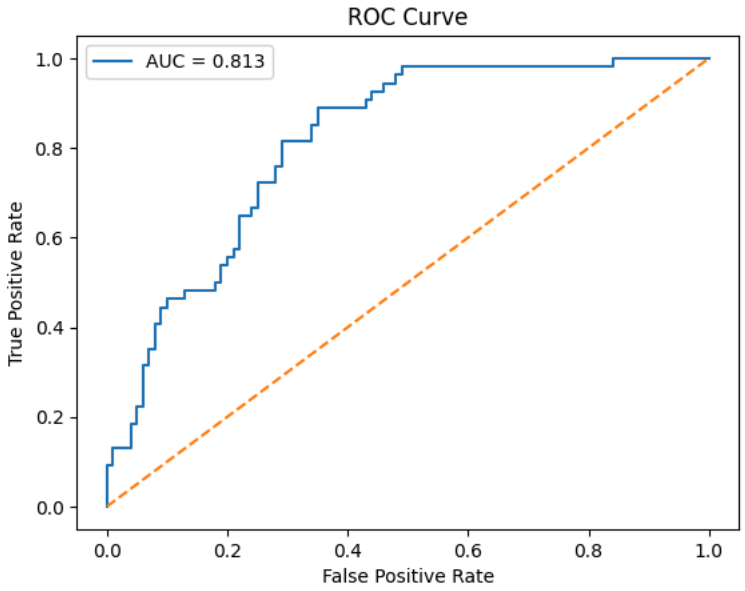
print("ROC-AUC:", auc)
```

```
ROC-AUC: 0.8125925925925926
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}")
plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```




```
features = df.drop("Outcome", axis=1).columns

coefficients = pd.DataFrame({
    "Feature": features,
```

```
    "Coefficient": model.coef_[0]
})

print(coefficients.sort_values(
    by="Coefficient",
    ascending=False
))
```

```
                    Feature  Coefficient
1                   Glucose     1.183444
5                       BMI     0.709709
0               Pregnancies     0.373019
6  DiabetesPedigreeFunction     0.287745
7                       Age     0.186393
3             SkinThickness     0.013877
2             BloodPressure    -0.014548
4                   Insulin    -0.044651
```

```
coefficients = coefficients.sort_values(
    by="Coefficient",
    ascending=True
)

plt.figure(figsize=(8, 5))
plt.barh(coefficients["Feature"], coefficients["Coefficient"])

plt.xlabel("Coefficient Value")
plt.ylabel("Features")
plt.title("Logistic Regression Feature Coefficients")

plt.axvline(x=0, linestyle="--")
plt.show()
```

